

EVIDENCIA VII

Alumno: Cristian Yanes De Lisio (LU: 1232006)

Temario: conjuntos: creación, pertenencia, operaciones y métodos. Diccionarios: pares clave-valor, acceso, alta, modificación, eliminación y recorridos. Selección de la estructura adecuada y aplicación al proyecto integrador.

1. Primer contacto con los conjuntos

Un conjunto (set) es una colección mutable de elementos únicos. No es una secuencia: sus elementos no se identifican mediante posiciones y, por lo tanto, no se accede a ellos con índices ni slicing. El orden de impresión no debe utilizarse como criterio lógico del programa.

Los elementos de un conjunto deben ser de un tipo de dato que no pueda modificarse una vez creado. Por ejemplo, pueden incluirse números, cadenas de texto o tuplas cuyos elementos también sean inmutables. En cambio, una lista no puede incorporarse como elemento de un conjunto porque su contenido puede modificarse.

Ejecuten el código y respondan:

- ¿Por qué los valores repetidos aparecen una sola vez?
- ¿Por qué un conjunto vacío se crea con set() y no con {}?
- ¿Sería correcto intentar mostrar lenguajes[0]? Justifiquen.

```

1  # Responder:
2  # a) ¿Por qué los valores repetidos aparecen una sola vez?
3  # b) ¿Por qué un conjunto vacío se crea con set() y no con {}?
4  # c) ¿Sería correcto intentar mostrar lenguajes[0]? Justifiquen.
5
6  lenguajes = {"Python", "Java", "C", "Python", "C"}
7
8  print("\n -----")
9  print(f"a) Porque debido a su estructura, un conjunto es una colección de elementos sin orden y sin duplicados. {lenguajes}")
10 print("----")
11
12 vacio1= {}
13 vacio2= set()
14
15 print(f"b) las llaves crean un diccionario ({type(vacio1)}), mientras que set() crea un conjunto ({type(vacio2)}) ")
16 print("----")
17
18 print("c) No, debido a que no es una secuencia: sus elementos no se identifican mediante posiciones y, por lo tanto, no se accede a ellos
19 try:
20     print(lenguajes[0])
21 except TypeError as error:
22     print(f"el error es: {error}")
23 print("----")
ic en gestión TI/2. Programación I/Evidencias/Clase 9/Ejercicio 1.py"
-----
a) Porque debido a su estructura, un conjunto es una colección de elementos sin orden y sin duplicados. {'Python', 'Java', 'C'}
---
b) las llaves crean un diccionario (<class 'dict'>), mientras que set() crea un conjunto (<class 'set'>)
---
c) No, debido a que no es una secuencia: sus elementos no se identifican mediante posiciones y, por lo tanto, no se accede a ellos con índices ni slicing.
el error es: 'set' object is not subscriptable
---
```

Resolver e interpretar cada resultado:

- a) Unión: grupo_a | grupo_b.
- b) Intersección: grupo_a & grupo_b.
- c) Diferencia: grupo_a - grupo_b y grupo_b - grupo_a.
- d) Diferencia simétrica: grupo_a ^ grupo_b.
- e) Pertenencia: 103 in grupo_a y 108 not in grupo_b.
- f) Explicar por qué la diferencia no es una operación conmutativa.

```
1 # 2. Pertenencia y operaciones entre conjuntos: resolver e interpretar cada resultado.
2 # a) Unión: grupo_a | grupo_b.
3 # b) Intersección: grupo_a & grupo_b.
4 # c) Diferencia: grupo_a - grupo_b y grupo_b - grupo_a.
5 # d) Diferencia simétrica: grupo_a ^ grupo_b.
6 # e) Pertenencia: 103 in grupo_a y 108 not in grupo_b.
7 # f) Explicar por qué la diferencia no es una operación conmutativa.
8 grupo_a = {101, 102, 103, 104, 105}
9 grupo_b = {104, 105, 106, 107}
10 # a)
11 print("...")
12 print(f'la union junta elementos de ambos conjuntos, si se repite algún elemento aparece una vez + {grupo_a | grupo_b}')
13 print("...")
14 print(f'la intersección toma solamente los elementos que estan en ambos conjuntos + {grupo_a & grupo_b}')
15 print("...")
16 print(f'la diferencia toma solamente los elementos que están en el primer conjunto pero no en el segundo conjunto comparado + {grupo_a - grupo_b}, {grupo_b - grupo_a}')
17 print("...")
18 print(f'la diferencia simetrica compara dos conjuntos para unirlos y quitar elementos que tengan en común + {grupo_a^grupo_b}')
19 print("...")
20 print(f'la pertenencia sirve para constatar si un elemento se encuentra dentro del conjunto con in/not in + 103 en el conjunto a:", 103 in grupo_a, "108 en el conjunto b:", 108 in grupo_b)
21 print("...")
22 print("porque el orden altera los resultados al igual que en una resta de números")
23 print("...")
24 conjunto_c = {"teo", "cristian", "joaquin", "e", 17}
25 conjunto_d = {"teo", "cristian", "joaquin", "e", 17}
26 print(f'la diferencia comun y simétrica devuelven conjuntos vacíos si tienen los mismos elementos: {conjunto_c - conjunto_d} {conjunto_c ^ conjunto_d}')
---
la union junta elementos de ambos conjuntos, si se repite algún elemento aparece una vez + {101, 102, 103, 104, 105, 106, 107}
---
la intersección toma solamente los elementos que estan en ambos conjuntos + {104, 105}
---
la diferencia toma solamente los elementos que están en el primer conjunto pero no en el segundo conjunto comparado + {101, 102, 103}, {106, 107}
---
la diferencia simetrica compara dos conjuntos para unirlos y quitar elementos que tengan en común + {101, 102, 103, 106, 107}
---
la pertenencia sirve para constatar si un elemento se encuentra dentro del conjunto con in/not in + 103 en el conjunto a: True 108 en el conjunto b: False
---
porque el orden altera los resultados al igual que en una resta de números
---
la diferencia comun y simétrica devuelven conjuntos vacíos si tienen los mismos elementos: set() set()
---
```

3. Métodos y relaciones entre conjuntos

Crear un conjunto llamado tecnologías con al menos cuatro elementos y realizar:

- a) Agregar una tecnología mediante add().
- b) Eliminar una tecnología existente mediante remove().
- c) Intentar eliminar una tecnología inexistente con remove() y registrar la excepción generada.
- d) Repetir la operación con discard() y comparar el comportamiento.
- e) Verificar con issubset() si las tecnologías obligatorias están incluidas en tecnologías.
- f) Verificar con issuperset() la relación inversa.
- g) Vaciar una copia del conjunto mediante clear(), sin perder el conjunto original.

```

3 TECNOLOGIAS = {"playstation", "mouse", "parlante", "reloj"}
4
5 print("----")
6 agregar_tecno= TECNOLOGIAS.add("router")
7 print(f'a) + agregar un elemento al conjunto mediante .add {TECNOLOGIAS}')
8 print("----")
9 quitar_tecno= TECNOLOGIAS.remove("reloj")
10 print(f'b) + quitar un elemento del conjunto mediante .remove {TECNOLOGIAS}')
11 print("----")
12 print('c) + quitar un elemento inexistente mediante .remove provoca un error de tipo KeyError: remove exige su existencia')
13 print("----")
14 quitar_discard= TECNOLOGIAS.discard("auricular")
15 print(f'd) + nquitar un elemento inexistente con el método discard no sucede nada y continúa la ejecución normal: {quitar_discard}')
16 print("----")
17
18 tecnologias2= {"playstation", "mouse", "parlante", "reloj"}
19 tecnologias3= {"playstation", "mouse", "parlante", "reloj", "auricular"}
20
21 print(f'e) + verificación con issubset() si un conjunto tecnologías esta incluido dentro de otro (todos sus elementos están dentro de este otro): {tecnologias2.issubset(tecnologias3)}')
22 print("tambien se puede chequear con un if else")
23 if tecnologias2.issubset(tecnologias3):
24 | print("el conjunto tecnologías2 está incluido dentro del conjunto tecnologías3")
25 else:
26 | print("no está contenido")
27 print("----")
28
29 print(f'f) + verificación con issuperset() la relación inversa: {tecnologias3.issuperset(tecnologias2)}')
30 print("----")
31 print("g) + vaciado de una copia del conjunto con clear() sin afectar al original:")
32 copia_a_vaciar = TECNOLOGIAS.copy()
33 copia_a_vaciar.clear()
34 print("----")
35 print(f'f) copia vaciada: {copia_a_vaciar} copia original: {TECNOLOGIAS}')
36
37 ---
38 a) → agregar un elemento al conjunto mediante .add {'router', 'reloj', 'playstation', 'parlante', 'televisor', 'mouse'}
39 ---
40 b) → quitar un elemento del conjunto mediante .remove {'router', 'playstation', 'parlante', 'televisor', 'mouse'}
41 ---
42
43 Traceback (most recent call last):
44   File "c:\Users\crisy\OneDrive\Escritorio\Lic en gestión TI\2. Programación I\Evidencias\Clase 9\Ejercicio 3.py", line 12, in <module>
45     quitar_inexistente=tecnologias.remove("auricular")
46   KeyError: 'auricular'
47
48 ---
49 a) → agregar un elemento al conjunto mediante .add {'playstation', 'router', 'mouse', 'parlante', 'reloj'}
50 ---
51 b) → quitar un elemento del conjunto mediante .remove {'playstation', 'router', 'mouse', 'parlante'}
52 ---
53 c) → quitar un elemento inexistente mediante .remove provoca un error de tipo KeyError: remove exige su existencia
54 ---
55 d) → nquitar un elemento inexistente con el método discard no sucede nada y continúa la ejecución normal: None
56 ---
57 e) → verificación con issubset() si un conjunto tecnologías esta incluido dentro de otro (todos sus elementos están dentro de este otro): True
58 tambien se puede chequear con un if else
59 el conjunto tecnologías2 está incluido dentro del conjunto tecnologías3
60 ---
61 f) → verificación con issuperset() la relación inversa: True
62 ---
63 g) → vaciado de una copia del conjunto con clear() sin afectar al original:
64 ---
65 copia vaciada: set() copia original: {'playstation', 'router', 'mouse', 'parlante'}
66 ---

```

4. Aplicación de conjuntos: eliminación de duplicados

Dada una lista de códigos de alumnos que puede contener repeticiones, desarrollen una función `codigos_unicos(codigos)` que retorne un conjunto con los códigos diferentes.

- a) Informen cuántos códigos distintos hay.
- b) Verifiquen si el código 105 fue informado.
- c) Conviertan el conjunto en una lista ordenada para mostrar un resultado predecible.
- d) Expliquen qué información se pierde al convertir directamente una lista en conjunto.

```

1  # 4. Aplicación de conjuntos
2  # Dada una lista de códigos de alumnos que puede contener repeticiones, desarrollen una f
3  # con los códigos diferentes.
4
5  codigos_alumnos= ["A03", "A07", "A048", "A056", "A017", "A046", "A03", "A07", "A056"]
6
7  def codigos_unicos(codigos):
8      return set(codigos_alumnos)
9
10 def informar_distintos(conjunto):
11     return len(conjunto)
12
13 resultado_unicos = codigos_unicos(codigos_alumnos)
14
15 def verificar_codigo(conjunto, codigo):
16     return codigo in conjunto
17
18 codigo_buscado= "105"
19
20 def convertir_conjunto_a_lista(conjunto):
21     return sorted(conjunto) #sorted devuelve y ordena una lista de A a Z
22
23 def convertir_lista_a_conjunto(lista):
24     return set(lista)
25
26 print("---")
27 print(f'lista convertida a un conjunto con códigos diferentes: {codigos_unicos(codigos_alumnos)}')
28 print("---")
29 print(f'a) → cantidad de códigos distintos: {informar_distintos(resultado_unicos)}')
30 print("---")
31 print(f'b) → verificar si el código 105 fue informado: {verificar_codigo(codigos_alumnos, codigo_buscado)}')
32 print("---")
33 print(f'c) → Conviertan el conjunto en una lista ordenada para mostrar un resultado predecible: {convertir_conjunto_a_lista(resultado_unicos)}')
34 print("---")
35 print(f'd) → ¿que info. se pierde al convertir una lista en conjunto? {convertir_lista_a_conjunto(codigos_alumnos)}')
36 print("1º se pierde la info. de elementos duplicados, 2º se pierde el orden original de los elementos")
37 print("---")

```

ic en gestión TI/2. Programación I/Evidencias/Clase 9/Ejercicio 4.py"

```

---
lista convertida a un conjunto con códigos diferentes: {'A017', 'A056', 'A07', 'A046', 'A03', 'A048'}
---
a) → cantidad de códigos distintos: 6
---
b) → verificar si el código 105 fue informado: False
---
c) → Conviertan el conjunto en una lista ordenada para mostrar un resultado predecible: ['A017', 'A03', 'A046', 'A048', 'A056', 'A07']
---
d) → ¿que info. se pierde al convertir una lista en conjunto? {'A017', 'A056', 'A07', 'A046', 'A03', 'A048'}
1º se pierde la info. de elementos duplicados, 2º se pierde el orden original de los elementos
---

```

5. Primer contacto con los diccionarios

Un diccionario (dict) representa una colección de pares clave-valor. Cada clave es única y debe ser hashable; los valores pueden ser de cualquier tipo y repetirse. El diccionario es mutable: permite agregar, modificar y eliminar pares. Sus valores se consultan mediante claves, no mediante posiciones.

- a) ¿Qué representa cada clave y cada valor?
- b) ¿Qué ocurriría si se volviera a asignar un valor a la clave 1002?

- c) ¿Por qué una lista no puede utilizarse como clave?
- d) ¿Qué diferencia existe entre {} y set()?

```

1 # 5. Primer contacto con los diccionarios (dict)
2
3 alumnos = {
4     1001: "Ana",
5     1002: "Bruno",
6     1003: "Carla"
7 }
8
9 diccionario_vacio = {}
10 conjunto_vacio = set()
11
12 print("---")
13 print(f'a) → en este diccionario, cada clave representa un código inmutable la cual permite acceder a un alumno en particular (valor) → {alumnos}')
14 alumnos[1002]="teo"
15 print(f'b) → que ocurre si se le vuelve a asignar un valor a la clave 1002: {alumnos} → el valor se reemplaza')
16 print('c) → porque una lista no puede utilizarse como clave: Las claves deben pertenecer a un tipo inmutable: números, cadenas o tuplas.')
17 print(f'd) → las llaves vacías indican la creación de un diccionario: {diccionario_vacio}, en cambio set() se utiliza para crear un conjunto: {conjunto_vacio}')
18
19 ---
20 a) → en este diccionario, cada clave representa un código inmutable la cual permite acceder a un alumno en particular (valor) → {1001: 'Ana', 1002: 'Bruno', 1003: 'Carla'}
21 ---
22 b) → que ocurre si se le vuelve a asignar un valor a la clave 1002: {1001: 'Ana', 1002: 'teo', 1003: 'Carla'} → el valor se reemplaza
23 ---
24 c) → porque una lista no puede utilizarse como clave: Las claves deben pertenecer a un tipo inmutable: números, cadenas o tuplas.
25 ---
26 d) → las llaves vacías indican la creación de un diccionario: {}, en cambio set() se utiliza para crear un conjunto: set()
27 ---
28 PS C:\Users\crisy\AppData\Local\Programs\Microsoft VS Code>

```

6. Acceso, alta, modificación y eliminación

Utilizando el diccionario alumnos, resolver:

- a) Mostrar el nombre asociado al legajo 1002 mediante alumnos[1002].
- b) Agregar el legajo 1004 y luego modificar el nombre asociado al legajo 1001.
- c) Verificar con in si existe el legajo 1010.
- d) Consultarlo con get() y un mensaje alternativo.
- e) Intentar acceder directamente a la clave inexistente y capturar KeyError.
- f) Eliminar un legajo existente con del, luego de comprobar su existencia.

El operador in aplicado a un diccionario consulta sus claves. get() permite obtener un valor sin generar KeyError cuando la clave no existe. La asignación diccionario[clave] = valor agrega un par nuevo o reemplaza el valor si la clave ya estaba registrada.

```

---
a) → mostrar el nombre (valor) asociado al legajo 1002: Bruno
---
b) → agregar el legajo 1004 y modificar el nombre (valor) asociado al legajo 1001: {1001: 'Luciana', 1002: 'Bruno', 1003: 'Carla', 1004: 'Teo'}
---
c) verificar con in si existe el legajo 1010:
no se ha encontrado el legajo
---
d) consultar con get() y un mensaje alternativo si no existe: no se ha encontrado el legajo 1010
Traceback (most recent call last):
  File "c:\Users\crisy\OneDrive\Escritorio\Lic en gestión TI\2. Programación I\Evidencias\Clase 9\Ejercicio 6.py", line 35, in <module>
    print(f'e) → intentar acceder a la clave inexistente y capturar el keyerror: {alumnos[2005]}')
    ~~~~~~^~~~~~
KeyError: 2005

```

```

3  alumnos = {
4      1001: "Ana",
5      1002: "Bruno",
6      1003: "Carla"
7  }
8
9  print("---")
10 print(f"a) → mostrar el nombre (valor) asociado al legajo 1002: {alumnos[1002]}")
11 print("---")
12
13 # b)
14 alumnos[1004] = "Teo"
15 alumnos[1001] = "Luciana"
16 print(f"b) → agregar el legajo 1004 y modificar el nombre (valor) asociado al legajo 1001: {alumnos}")
17 print("---")
18 print("c) verificar con in si existe el legajo 1010:")
19 if 1010 in alumnos:
20     print("legajo encontrado")
21 else:
22     print("no se ha encontrado el legajo")
23 print("---")
24 resultado_get = alumnos.get(1010, "no se ha encontrado el legajo 1010")
25 print(f"d) consultar con get() y un mensaje alternativo si no existe: {resultado_get}")
26 print("---")
27 if 1003 in alumnos:
28     del alumnos[1003]
29 print(f"f) → eliminar un legajo inexistente con del, luego comprobar su existencia {alumnos}')
---
```

a) → mostrar el nombre (valor) asociado al legajo 1002: Bruno

b) → agregar el legajo 1004 y modificar el nombre (valor) asociado al legajo 1001: {1001: 'Luciana', 1002: 'Bruno', 1003: 'Carla', 1004: 'Teo'}

c) verificar con in si existe el legajo 1010:
no se ha encontrado el legajo

d) consultar con get() y un mensaje alternativo si no existe: no se ha encontrado el legajo 1010

f) → eliminar un legajo inexistente con del, luego comprobar su existencia {1001: 'Luciana', 1002: 'Bruno', 1004: 'Teo'}

7. Recorridos de diccionarios

- a) Recorrer directamente el diccionario y mostrar sus claves.
- b) Mostrar las cantidades mediante values().
- c) Mostrar cada producto y su stock mediante items().
- d) Informar la cantidad de productos diferentes con len().
- e) Calcular el total de unidades disponibles.
- f) Informar el producto con mayor stock mediante max(stock, key=stock.get).

La expresión `max(stock.values())` devuelve la mayor cantidad, pero no identifica el producto. Para obtener la clave asociada al mayor valor puede utilizarse `max(stock, key=stock.get)`. Analicen qué ocurriría si el diccionario estuviera vacío o si dos productos compartieran la cantidad máxima.

```

1 # 7. Recorridos de diccionarios
2
3 stock = {
4     "teclado": 12,
5     "mouse": 20,
6     "monitor": 7,
7     "webcam": 5
8 }
9
10 print("a) recorrer el diccionario y mostrar sus claves")
11 for clave in stock:
12     print(f'claves: {clave}')
13 print("----")
14 print(f'b) mostrar las cantidades mediante values(): {list(stock.values())}') #agregar el list fuera para evitar dict_values (view object)
15 print("----")
16 print(f'c) Mostrar cada producto y su stock mediante items(): {list(stock.items())}')
17 print("----")
18 print(f'd) informar la cantidad de productos diferentes con len(): {len(stock)} ')
19 print("----")
20 total_unidades=sum(stock.values())
21 print(f'e) calcular el total de unidades disponibles: {total_unidades} ')
22 print("----")
23 producto_mayor_stock=max(stock, key=stock.get)
24 cantidad_mayor_unidades = stock[producto_mayor_stock]
25 print(f'f) informar el producto con mayor stock: el producto es {producto_mayor_stock} que cuenta con {cantidad_mayor_unidades} unidades')
26 print("----")

```

a) recorrer el diccionario y mostrar sus claves
claves: teclado
claves: mouse
claves: monitor
claves: webcam

b) mostrar las cantidades mediante values(): [12, 20, 7, 5]

c) Mostrar cada producto y su stock mediante items(): [('teclado', 12), ('mouse', 20), ('monitor', 7), ('webcam', 5)].

d) informar la cantidad de productos diferentes con len(): 4

e) calcular el total de unidades disponibles: 44

f) informar el producto con mayor stock: el producto es mouse que cuenta con 20 unidades

8. Diccionarios con valores compuestos

Crear un diccionario cuya clave sea el nombre de un grupo y cuyo valor sea un conjunto con las tecnologías que utiliza. Esta estructura combina dos niveles: el diccionario localiza al grupo y el conjunto evita tecnologías repetidas.

- a) Agreguen un nuevo grupo.
- b) Incorporen una tecnología a un grupo existente.
- c) Consulten un grupo mediante get() sin provocar una excepción.
- d) Recorrer el diccionario mostrando cada grupo y sus tecnologías ordenadas solo para la presentación.

```

1 # 8. Diccionarios con valores compuestos
2
3 # Crear un diccionario cuya clave sea el nombre de un grupo y cuyo valor sea un conjunto con las tecnologías que utiliza.
4 # Esta estructura combina dos niveles: el diccionario localiza al grupo y el conjunto evita tecnologías repetidas.
5
6 tecnologias_por_grupo = {
7     "Grupo 1": {"Python", "Git"},
8     "Grupo 2": {"Python", "JSON"}
9 }
10
11 tecnologias_por_grupo["Grupo 3"] = {"Html", "Css"} # si no uso llaves se crea una tupla dentro del conjunto
12 print("---")
13 print(f'a) → agregar un nuevo grupo al diccionario: {tecnologias_por_grupo}')
14 print("---")
15 tecnologias_por_grupo["Grupo 3"].add("javascript")
16 print(f'b) → incorporar una tecnología a un grupo existente: {tecnologias_por_grupo}')
17 print("---")
18 consultar_grupo = tecnologias_por_grupo.get("Grupo 3")
19 print(f'c) → consultar un grupo mediante get() sin provocar una excepción {consultar_grupo}')
20 print("---")
21 print(f'd) → recorrer el diccionario mostrando cada grupo y sus tecnologías')
22 for grupo in tecnologias_por_grupo:
23     print(f'{grupo} {tecnologias_por_grupo[grupo]}')
24
25 ---
26 a) → agregar un nuevo grupo al diccionario: {'Grupo 1': {'Python', 'Git'}, 'Grupo 2': {'Python', 'JSON'}, 'Grupo 3': {'Css', 'Html'}}
27 ---
28 b) → incorporar una tecnología a un grupo existente: {'Grupo 1': {'Python', 'Git'}, 'Grupo 2': {'Python', 'JSON'}, 'Grupo 3': {'javascript', 'Css', 'Html'}}
29 ---
30 c) → consultar un grupo mediante get() sin provocar una excepción {'javascript', 'Css', 'Html'}
31 ---
32 d) → recorrer el diccionario mostrando cada grupo y sus tecnologías
33 Grupo 1 {'Python', 'Git'}
34 Grupo 2 {'Python', 'JSON'}
35 Grupo 3 {'javascript', 'Css', 'Html'}
36 ---

```

9. Actividad integradora: análisis de una frase

Desarrollar un programa que solicite una frase y realice las siguientes tareas:

- a) Normalizarla a minúsculas.
- b) Eliminar, como mínimo, punto, coma, punto y coma, dos puntos, signos de pregunta y exclamación.
- c) Separarla en palabras.
- d) Crear un conjunto con las palabras diferentes.
- e) Crear un diccionario de frecuencias donde cada palabra sea una clave y su cantidad de apariciones sea el valor.
- f) Mostrar las palabras sin repetir en orden alfabético.
- g) Mostrar cada palabra con su frecuencia.
- h) Informar la palabra más frecuente solo si se ingresó al menos una palabra.

Prueben con una frase que contenga palabras repetidas, diferencias de mayúsculas y signos de puntuación. Incluyan también una cadena vacía o formada solo por espacios para evitar aplicar max() a un diccionario vacío.

```

1  # 9. Actividad integradora: análisis de una frase
2  # Desarrollar un programa que solicite una frase y realice las siguientes tareas:
3
4  import funciones
5
6  frase_solicitada = funciones.solicitar_frase()
7  print("---")
8  frase_normalizada = funciones.normalizar_frase(frase_solicitada)
9  print("---")
10 frase_sin_signos = funciones.eliminar_caracteres(frase_normalizada)
11 print("---")
12 palabras = funciones.separar_palabras(frase_sin_signos)
13 print("---")
14 conjunto_palabras = funciones.crear_conjunto(palabras)
15 print("---")
16 frecuencias = funciones.crear_frecuencias(palabras)
17 print("---")
18 funciones.mostrar_orden_alfabetico(conjunto_palabras)
19 print("---")
20 funciones.mostrar_frecuencias(frecuencias)
21 print("---")
22 funciones.mostrar_mas_frecuente(frecuencias)
23 print("---")
24
25 def solicitar_frase():
26     frase = input("Ingrese una frase: ")
27     while frase.strip() == "":
28         print("la frase no puede estar vacía ni contener solamente espacios.")
29         frase = input("ingrese una frase nuevamente: ")
30     return frase
31
32 def normalizar_frase(frase):
33     frase_normalizada = frase.lower()
34     print(f'a) → frase normalizada: {frase_normalizada}')
35     return frase_normalizada
36
37 def eliminar_caracteres(frase):
38     frase_sin_signos = (
39         frase.replace(".", "")
40         .replace(",", "")
41         .replace(";", "")
42         .replace(":", "")
43         .replace("?", "")
44         .replace(";", "")
45         .replace("!", "")
46         .replace("!", "")
47     )
48     print(f'b) → frase sin signos: {frase_sin_signos}')
49     return frase_sin_signos
50
51 def separar_palabras(frase):
52     palabras = frase.split()
53     print(f'c) → palabras: {palabras}')
54     return palabras
55
56 def crear_conjunto(palabras):
57     conjunto_palabras = set(palabras)
58     print(f'd) → palabras diferentes: {conjunto_palabras}')
59     return conjunto_palabras
60

```

```

41
42 def crear_frecuencias(palabras):
43     frecuencias = {}
44     for palabra in palabras:
45         frecuencias[palabra] = frecuencias.get(palabra, 0) + 1
46     print(f'e) → diccionario de frecuencias: {frecuencias}')
47     return frecuencias
48
49
50 def mostrar_orden_alfabetico(conjunto_palabras):
51     palabras_ordenadas = sorted(conjunto_palabras)
52     print(f'f) → palabras sin repetir en orden alfabético:')
53     for palabra in palabras_ordenadas:
54         print(f' {palabra}')
55
56 def mostrar_frecuencias(frecuencias):
57     print("g) → frecuencia de cada palabra:")
58     for palabra, frecuencia in frecuencias.items():
59         print(f'{palabra}: {frecuencia}')
60
61
62 def mostrar_mas_frecuente(frecuencias):
63     if len(frecuencias) > 0:
64         palabra_mas_frecuente = max(frecuencias, key=frecuencias.get)
65         frecuencia = frecuencias[palabra_mas_frecuente]
66         print(f'h) → palabra más frecuente: {palabra_mas_frecuente}')
67         print(f'aparece {frecuencia} veces.')
68     else:
69         print("no se ingresaron palabras.")
70
71 ramacion 1/tvidencias/Clase 9/Ejercicio 9.py
Ingrese una frase: ¿¿Juan Juan ! de los palotes.
---
a) → frase normalizada: ¿¿juan juan ! de los palotes.
---
b) → frase sin signos: juan juan de los palotes
---
c) → palabras: ['juan', 'juan', 'de', 'los', 'palotes']
---
d) → palabras diferentes: {'palotes', 'de', 'juan', 'los'}
---
e) → diccionario de frecuencias: {'juan': 2, 'de': 1, 'los': 1, 'palotes': 1}
---
f) → palabras sin repetir en orden alfabético:
de
juan
los
palotes
---
g) → frecuencia de cada palabra:
juan: 2
de: 1
los: 1
palotes: 1
---
h) → palabra más frecuente: juan
aparece 2 veces.
---

```